

Выделение и освобождение номеров устройств

```
extern int alloc_chrdev_region(dev_t *, unsigned, unsigned, const char *);
extern int register_chrdev_region(dev_t, unsigned, const char *);
extern int __register_chrdev(unsigned int major, unsigned int baseminor,
                             unsigned int count, const char *name,
                             const struct file_operations *fops);
extern void __unregister_chrdev(unsigned int major, unsigned int
                                baseminor, unsigned int count, const char *name);
extern void unregister_chrdev_region(dev_t, unsigned);
extern void chrdev_show(struct seq_file *, off_t);

static inline int register_chrdev(unsigned int major, const char *name,
                                   const struct file_operations *fops)
{
    return __register_chrdev(major, 0, 256, name, fops);
}

static inline void unregister_chrdev(unsigned int major, const char *name)
{
    __unregister_chrdev(major, 0, 256, name);
}
```

Листинг 0 — Выделение и освобождение номеров устройств

Обработчики аппаратных прерываний

```
typedef irqreturn_t (*irq_handler_t)(int, void *);

static inline int __must_check
request_irq(unsigned int irq, irq_handler_t handler, unsigned long flags,
            const char *name, void *dev)
{
    return request_threaded_irq(irq, handler, NULL, flags |
    IRQF_COND_ONESHOT, name, dev);
}
```

Листинг 1 — Обработчики аппаратных прерываний

Флаги IRQ

```
/*
 * These flags used only by the kernel as part of the
 * irq handling routines.
 *
 * IRQF_SHARED - allow sharing the irq among several devices
 * IRQF_PROBE_SHARED - set by callers when they expect sharing mismatches
to occur
 * IRQF_TIMER - Flag to mark this interrupt as timer interrupt
 * IRQF_PERCPU - Interrupt is per cpu
 * IRQF_NOBALANCING - Flag to exclude this interrupt from irq balancing
 * IRQF_IRQPOLL - Interrupt is used for polling (only the interrupt that
is
 *
                registered first in a shared interrupt is considered for
 *
                performance reasons)
 * IRQF_ONESHOT - Interrupt is not reenabled after the hardirq handler
```

```

finished.
*           Used by threaded interrupts which need to keep the
*           irq line disabled until the threaded handler has been
run.
* IRQF_NO_SUSPEND - Do not disable this IRQ during suspend. Does not
guarantee
*           that this interrupt will wake the system from a
suspended
*           state. See Documentation/power/suspend-and-
interrupts.rst
* IRQF_FORCE_RESUME - Force enable it on resume even if IRQF_NO_SUSPEND
is set
* IRQF_NO_THREAD - Interrupt cannot be threaded
* IRQF_EARLY_RESUME - Resume IRQ early during syscore instead of at
device
*           resume time.
* IRQF_COND_SUSPEND - If the IRQ is shared with a NO_SUSPEND user,
execute this
*           interrupt handler after suspending interrupts. For
system
*           wakeup devices users need to implement wakeup detection
in
*           their interrupt handlers.
* IRQF_NO_AUTOEN - Don't enable IRQ or NMI automatically when users
request it.
*           Users will enable it explicitly by enable_irq() or
enable_nmi()
*           later.
* IRQF_NO_DEBUG - Exclude from runaway detection for IPI and similar
handlers,
*           depends on IRQF_PERCPU.
* IRQF_COND_ONESHOT - Agree to do IRQF_ONESHOT if already set for a
shared
*           interrupt.
*/
#define IRQF_SHARED      0x00000080
#define IRQF_PROBE_SHARED 0x00000100
#define __IRQF_TIMER      0x00000200
#define IRQF_PERCPU      0x00000400
#define IRQF_NOBALANCING  0x00000800
#define IRQF_IRQPOLL      0x00001000
#define IRQF_ONESHOT      0x00002000
#define IRQF_NO_SUSPEND   0x00004000
#define IRQF_FORCE_RESUME 0x00008000
#define IRQF_NO_THREAD    0x00010000
#define IRQF_EARLY_RESUME 0x00020000
#define IRQF_COND_SUSPEND 0x00040000
#define IRQF_NO_AUTOEN    0x00080000
#define IRQF_NO_DEBUG     0x00100000
#define IRQF_COND_ONESHOT 0x00200000

```

Листинг 2 — Флаги IRQ

struct irqaction

```
/**
 * struct irqaction - per interrupt action descriptor
 * @handler: interrupt handler function
 * @name: name of the device
 * @dev_id: cookie to identify the device
 * @percpu_dev_id: cookie to identify the device
 * @affinity: CPUs this irqaction is allowed to run on
 * @next: pointer to the next irqaction for shared interrupts
 * @irq: interrupt number
 * @flags: flags (see IRQF_* above)
 * @thread_fn: interrupt handler function for threaded interrupts
 * @thread: thread pointer for threaded interrupts
 * @secondary: pointer to secondary irqaction (force threading)
 * @thread_flags: flags related to @thread
 * @thread_mask: bitmask for keeping track of @thread activity
 * @dir: pointer to the proc/irq/NN/name entry
 */
struct irqaction {
    irq_handler_t handler;
    union {
        void *dev_id;
        void __percpu *percpu_dev_id;
    };
    const struct cpumask *affinity;
    struct irqaction *next;
    irq_handler_t thread_fn;
    struct task_struct *thread;
    struct irqaction *secondary;
    unsigned int irq;
    unsigned int flags;
    unsigned long thread_flags;
    unsigned long thread_mask;
    const char *name;
    struct proc_dir_entry *dir;
}
```

Листинг 3 — struct irqaction

enum irqreturn

```
/**
 * enum irqreturn - irqreturn type values
 * @IRQ_NONE: interrupt was not from this device or was not handled
 * @IRQ_HANDLED: interrupt was handled by this device
 * @IRQ_WAKE_THREAD: handler requests to wake the handler thread
 */
enum irqreturn {
    IRQ_NONE = (0 << 0),
    IRQ_HANDLED = (1 << 0),
    IRQ_WAKE_THREAD = (1 << 1),
};
```

Листинг 4 — enum irqreturn

softirq

```
/* PLEASE, avoid to allocate new softirqs, if you need not _really_ high
   frequency threaded job scheduling. For almost all the purposes
   tasklets are more than enough. F.e. all serial device BHs et
   al. should be converted to tasklets, not to softirqs.
*/

enum
{
    HI_SOFTIRQ=0,
    TIMER_SOFTIRQ,
    NET_TX_SOFTIRQ,
    NET_RX_SOFTIRQ,
    BLOCK_SOFTIRQ,
    IRQ_POLL_SOFTIRQ,
    TASKLET_SOFTIRQ,
    SCHED_SOFTIRQ,
    HRTIMER_SOFTIRQ,
    RCU_SOFTIRQ,    /* Preferable RCU should always be the last softirq */

    NR_SOFTIRQS
};
```

Листинг 5 — softirq

tasklet

```
/* Tasklets --- multithreaded analogue of BHs.

   This API is deprecated. Please consider using threaded IRQs instead:
   https://lore.kernel.org/lkml/20200716081538.2sivhkj4hcyrusem@linutronix.de

   Main feature differing them of generic softirqs: tasklet
   is running only on one CPU simultaneously.

   Main feature differing them of BHs: different tasklets
   may be run simultaneously on different CPUs.

   Properties:
   * If tasklet_schedule() is called, then tasklet is guaranteed
     to be executed on some cpu at least once after this.
   * If the tasklet is already scheduled, but its execution is still not
     started, it will be executed only once.
   * If this tasklet is already running on another CPU (or schedule is
     called from tasklet itself), it is rescheduled for later.
   * Tasklet is strictly serialized wrt itself, but not
     wrt another tasklets. If client needs some intertask synchronization,
     he makes it with spinlocks.
*/

struct tasklet_struct
{
    struct tasklet_struct *next;
```

```

    unsigned long state;
    atomic_t count;
    bool use_callback;
    union {
        void (*func)(unsigned long data);
        void (*callback)(struct tasklet_struct *t);
    };
    unsigned long data;
};

#define DECLARE_TASKLET(name, _callback) \
struct tasklet_struct name = { \
    .count = ATOMIC_INIT(0), \
    .callback = _callback, \
    .use_callback = true, \
}

#define DECLARE_TASKLET_DISABLED(name, _callback) \
struct tasklet_struct name = { \
    .count = ATOMIC_INIT(1), \
    .callback = _callback, \
    .use_callback = true, \
}

#define from_tasklet(var, callback_tasklet, tasklet_fieldname) \
    container_of(callback_tasklet, typeof(*var), tasklet_fieldname)

#define DECLARE_TASKLET_OLD(name, _func) \
struct tasklet_struct name = { \
    .count = ATOMIC_INIT(0), \
    .func = _func, \
}

#define DECLARE_TASKLET_DISABLED_OLD(name, _func) \
struct tasklet_struct name = { \
    .count = ATOMIC_INIT(1), \
    .func = _func, \
}

enum
{
    TASKLET_STATE_SCHED, /* Tasklet is scheduled for execution */
    TASKLET_STATE_RUN /* Tasklet is running (SMP only) */
};

void tasklet_init(struct tasklet_struct *t,
    void (*func)(unsigned long), unsigned long data)
{
    t->next = NULL;
    t->state = 0;
    atomic_set(&t->count, 0);
    t->func = func;
    t->use_callback = false;
    t->data = data;
}

```

```

static void __tasklet_schedule_common(struct tasklet_struct *t,
                                     struct tasklet_head __percpu *headp,
                                     unsigned int softirq_nr)
{
    struct tasklet_head *head;
    unsigned long flags;

    local_irq_save(flags);
    head = this_cpu_ptr(headp);
    t->next = NULL;
    *head->tail = t;
    head->tail = &(t->next);
    raise_softirq_irqoff(softirq_nr);
    local_irq_restore(flags);
}

void __tasklet_schedule(struct tasklet_struct *t)
{
    __tasklet_schedule_common(t, &tasklet_vec,
                              TASKLET_SOFTIRQ);
}
EXPORT_SYMBOL(__tasklet_schedule);

void __tasklet_hi_schedule(struct tasklet_struct *t)
{
    __tasklet_schedule_common(t, &tasklet_hi_vec,
                              HI_SOFTIRQ);
}
EXPORT_SYMBOL(__tasklet_hi_schedule);

```

Листинг 6 — tasklet

struct worker

```

struct worker_pool;

/*
 * The poor guys doing the actual heavy lifting. All on-duty workers are
 * either serving the manager role, on idle list or on busy hash. For
 * details on the locking annotation (L, I, X...), refer to workqueue.c.
 *
 * Only to be used in workqueue and async.
 */
struct worker {
    /* on idle list while idle, on busy hash table while busy */
    union {
        struct list_head entry; /* L: while idle */
        struct hlist_node hentry; /* L: while busy */
    };

    struct work_struct *current_work; /* K: work being processed and its
    */
    work_func_t current_func; /* K: function */
    struct pool_workqueue *current_pwq; /* K: pwq */

```

```

u64    current_at; /* K: runtime at start or last wakeup */
unsigned long    current_start; /* K: start time of current work item */
unsigned int     current_color; /* K: color */

int     sleeping; /* S: is worker sleeping? */

/* used by the scheduler to determine a worker's last known identity */
work_func_t    last_func; /* K: last work's fn */

struct list_head    scheduled; /* L: scheduled works */

struct task_struct    *task; /* I: worker task */
struct worker_pool    *pool; /* A: the associated pool */
/* L: for rescuers */
struct list_head    node; /* A: anchored at pool->workers */
/* A: runs through worker->node */

unsigned long    last_active; /* K: last active timestamp */
unsigned int     flags; /* L: flags */
int     id; /* I: worker id */

/*
 * Opaque string set with work_set_desc(). Printed out with task
 * dump for debugging - WARN, BUG, panic or sysrq.
 */
char    desc[WORKER_DESC_LEN];

/* used only by rescuers to point to the target workqueue */
struct workqueue_struct    *rescue_wq; /* I: the workqueue to rescue */
};

```

ЛИСТИНГ 7 — struct worker

struct worker_pool

```

struct worker_pool {
    raw_spinlock_t    lock; /* the pool lock */
    int     cpu; /* I: the associated cpu */
    int     node; /* I: the associated node ID */
    int     id; /* I: pool ID */
    unsigned int     flags; /* L: flags */

    unsigned long    last_progress_ts; /* L: last forward progress timestamp */
    /*
     * bool    cpu_stall; /* WD: stalled cpu bound pool */

    /*
     * The counter is incremented in a process context on the associated CPU
     * w/ preemption disabled, and decremented or reset in the same context
     * but w/ pool->lock held. The readers grab pool->lock and are
     * guaranteed to see if the counter reached zero.
     */
    int     nr_running;

    struct list_head    worklist; /* L: list of pending works */

```

```

int      nr_workers; /* L: total number of workers */
int      nr_idle;    /* L: currently idle workers */

struct list_head idle_list; /* L: list of idle workers */
struct timer_list idle_timer; /* L: worker idle timeout */
struct work_struct idle_cull_work; /* L: worker idle cleanup */

struct timer_list mayday_timer; /* L: SOS timer for workers */

/* a workers is either on busy_hash or idle_list, or the manager */
DECLARE_HASHTABLE(busy_hash, BUSY_WORKER_HASH_ORDER);
/* L: hash of busy workers */

struct worker *manager; /* L: purely informational */
struct list_head workers; /* A: attached workers */

struct ida worker_ida; /* worker IDs for task name */

struct workqueue_attrs *attrs; /* I: worker attributes */
struct hlist_node hash_node; /* PL: unbound_pool_hash node */
int refcnt; /* PL: refcnt for unbound pools */
#ifdef CONFIG_PREEMPT_RT
spinlock_t cb_lock; /* BH worker cancel lock */
#endif
/*
 * Destruction of pool is RCU protected to allow dereferences
 * from get_work_pool().
 */
struct rcu_head rcu;
};

```

Листинг 8 — struct worker_pool

Инициализация очереди работ

```

#define DECLARE_WORK(n, f) \
    struct work_struct n = __WORK_INITIALIZER(n, f)

#define __INIT_WORK(_work, _func, _onstack) \
    do { \
        static __maybe_unused struct lock_class_key __key; \
        __INIT_WORK_KEY(_work, _func, _onstack, &__key); \
    } while (0)

#define INIT_WORK(_work, _func) \
    __INIT_WORK((_work), (_func), 0)

int queue_work(struct workqueue_struct *wq, struct work_struct *work);
int queue_work_on(int cpu, struct workqueue_struct *wq, struct work_struct *work);
int queue_delayed_work(struct workqueue_struct *wq, struct work_struct *work, unsigned long delay);

```

Листинг 9 — Инициализация очереди работ

struct socket

```
/**
 * enum sock_type - Socket types
 * @SOCK_STREAM: stream (connection) socket
 * @SOCK_DGRAM: datagram (conn.less) socket
 * @SOCK_RAW: raw socket
 * @SOCK_RDM: reliably-delivered message
 * @SOCK_SEQPACKET: sequential packet socket
 * @SOCK_DCCP: Datagram Congestion Control Protocol socket
 * @SOCK_PACKET: linux specific way of getting packets at the dev level.
 *               For writing rarp and other similar things on the user level.
 *
 * When adding some new socket type please
 * grep ARCH_HAS_SOCKET_TYPE include/asm-* /socket.h, at least MIPS
 * overrides this enum for binary compat reasons.
 */
enum sock_type {
    SOCK_STREAM = 1,
    SOCK_DGRAM  = 2,
    SOCK_RAW    = 3,
    SOCK_RDM    = 4,
    SOCK_SEQPACKET = 5,
    SOCK_DCCP   = 6,
    SOCK_PACKET = 10,
};

struct socket {
    socket_state    state;

    short          type;

    unsigned long   flags;

    struct file     *file;
    struct sock     *sk;
    const struct proto_ops *ops; /* Might change with IPV6_ADDRFORM or
    MPTCP. */

    struct socket_wq wq;
};
```

Листинг 10 — struct socket

Сокеты AF_INET, сетевой стек

```
struct in_addr {
    __be32    s_addr;
};

/* Structure describing an Internet (IP) socket address. */
#if __UAPI_DEF_SOCKADDR_IN
#define __SOCK_SIZE__ 16 /* sizeof(struct sockaddr) */
struct sockaddr_in {
    __kernel_sa_family_t  sin_family; /* Address family */
```

```

__be16    sin_port; /* Port number      */
struct in_addr sin_addr; /* Internet address */

/* Pad to size of `struct sockaddr'. */
unsigned char __pad[__SOCK_SIZE__ - sizeof(short int) -
    sizeof(unsigned short int) - sizeof(struct in_addr)];
};

```

Листинг 11 — Сокеты AF_INET, сетевой стек

sys_socketcall

```

asmlinkage long sys_socketcall(int call, unsigned long *args)
{
    int err;
    if (copy_from_user(a, args, args[call]))
        return -EFAULT;

    a0 = a[0];
    a1 = a[1];

    switch (call)
    {
        case SYS_SOCKET:
            err = sys_socket(a0, a1, a[2]);
            break;
        case SYS_BIND:
            err = sys_bind(a0, (struct sockaddr *) a1, a[2]);
            break;
        case SYS_CONNECT:
            err = sys_connect(a0, (struct sockaddr *) a1, a[2]);
            break;
        ...
    }

    return err;
}

```

Листинг 12 — sys_socketcall

sys_socket

```

asmlinkage long sys_socket(int family, int type, int protocol)
{
    int retval;
    struct socket *sock;

    ...
    retval = socket_create(family, type, protocol, &sock);
    ...
    return retval;
}

```

Листинг 13 — sys_socket

struct proc_dir_entry

```
struct proc_dir_entry {
    /*
     * number of callers into module in progress;
     * negative -> it's going away RSN
     */
    atomic_t in_use;
    refcount_t refcnt;
    struct list_head pde_openers; /* who did ->open, but not ->release */
    /* protects ->pde_openers and all struct pde_opener instances */
    spinlock_t pde_unload_lock;
    struct completion *pde_unload_completion;
    const struct inode_operations *proc_iops;
    union {
        const struct proc_ops *proc_ops;
        const struct file_operations *proc_dir_ops;
    };
    union {
        const struct seq_operations *seq_ops;
        int (*single_show)(struct seq_file *, void *);
    };
    proc_write_t write;
    void *data;
    unsigned int state_size;
    unsigned int low_ino;
    nlink_t nlink;
    kuid_t uid;
    kgid_t gid;
    loff_t size;
    struct proc_dir_entry *parent;
    struct rb_root subdir;
    struct rb_node subdir_node;
    char *name;
    umode_t mode;
    u8 flags;
    u8 namelen;
    char inline_name[];
}
```

Листинг 14 — struct proc_dir_entry

struct proc_ops

```
extern struct proc_dir_entry *proc_create_data(const char *, umode_t,
                                              struct proc_dir_entry *,
                                              const struct proc_ops *,
                                              void *);

struct proc_dir_entry *proc_create(const char *name, umode_t mode, struct
proc_dir_entry *parent, const struct proc_ops *proc_ops);

extern struct proc_dir_entry *proc_mkdir(const char *, struct
proc_dir_entry *);
```

```

extern struct proc_dir_entry *proc_symlink(const char *,
    struct proc_dir_entry *, const char *);

extern void remove_proc_entry(const char *, struct proc_dir_entry *);

struct proc_ops {
    unsigned int proc_flags;
    int (*proc_open)(struct inode *, struct file *);
    ssize_t (*proc_read)(struct file *, char __user *, size_t, loff_t *);
    ssize_t (*proc_read_iter)(struct kiocb *, struct iov_iter *);
    ssize_t (*proc_write)(struct file *, const char __user *, size_t, loff_t
*);
    /* mandatory unless nonseekable_open() or equivalent is used */
    loff_t (*proc_lseek)(struct file *, loff_t, int);
    int (*proc_release)(struct inode *, struct file *);
    __poll_t (*proc_poll)(struct file *, struct poll_table_struct *);
    long (*proc_ioctl)(struct file *, unsigned int, unsigned long);
#ifdef CONFIG_COMPAT
    long (*proc_compat_ioctl)(struct file *, unsigned int, unsigned long);
#endif
    int (*proc_mmap)(struct file *, struct vm_area_struct *);
    unsigned long (*proc_get_unmapped_area)(struct file *, unsigned long,
unsigned long, unsigned long, unsigned long);
} __randomize_layout;

static const struct proc_ops device_proc_ops = {
    .proc_open = device_open,
    .proc_read = device_read,
    .proc_write = device_write,
    .proc_release = device_release,
};

```

Листинг 15 — struct proc_ops

file_system_type

```

struct file_system_type {
    const char *name;
    int fs_flags;
#define FS_REQUIRES_DEV 1
#define FS_BINARY_MOUNTDATA 2
#define FS_HAS_SUBTYPE 4
#define FS_USERNS_MOUNT 8 /* Can be mounted by userns root */
#define FS_DISALLOW_NOTIFY_PERM 16 /* Disable fanotify permission events
*/
#define FS_ALLOW_IDMAP 32 /* FS has been updated to handle
vfs idmappings. */
#define FS_MGMTIME 64 /* FS uses multigrain timestamps */
#define FS_LBS 128 /* FS supports LBS */
#define FS_POWER_FREEZE 256 /* Always freeze on suspend/hibernate */
#define FS_RENAME_DOES_D_MOVE 32768 /* FS will handle d_move() during
rename() internally. */
    int (*init_fs_context)(struct fs_context *);
    const struct fs_parameter_spec *parameters;
    void (*kill_sb)(struct super_block *);
    struct module *owner;
};

```

```

struct file_system_type * next;
struct hlist_head fs_supers;

struct lock_class_key s_lock_key;
struct lock_class_key s_umount_key;
struct lock_class_key s_vfs_rename_key;
struct lock_class_key s_writers_key[SB_FREEZE_LEVELS];

struct lock_class_key i_lock_key;
struct lock_class_key i_mutex_key;
struct lock_class_key invalidate_lock_key;
struct lock_class_key i_mutex_dir_key;
};

```

Листинг 16 — file_system_type

struct super_block

```

struct super_block {
    struct list_head    s_list;    /* Keep this first */
    dev_t               s_dev;      /* search index; _not_ kdev_t */
    unsigned char       s_blocksize_bits;
    unsigned long       s_blocksize;
    loff_t              s_maxbytes; /* Max file size */
    struct file_system_type *s_type;
    const struct super_operations *s_op;
    const struct dquot_operations *dq_op;
    const struct quotactl_ops *s_qcop;
    const struct export_operations *s_export_op;
    unsigned long       s_flags;
    unsigned long       s_iflags; /* internal SB_I_* flags */
    unsigned long       s_magic;
    struct dentry       *s_root;
    struct rw_semaphore s_umount;
    int                 s_count;
    atomic_t            s_active;
#ifdef CONFIG_SECURITY
    void                *s_security;
#endif
    const struct xattr_handler *const *s_xattr;
#ifdef CONFIG_FS_ENCRYPTION
    const struct fscrypt_operations *s_cop;
    struct fscrypt_keyring *s_master_keys; /* master crypto keys in use */
#endif
#ifdef CONFIG_FS_VERITY
    const struct fsverity_operations *s_vop;
#endif
#ifdef IS_ENABLED(CONFIG_UNICODE)
    struct unicode_map *s_encoding;
    __u16               s_encoding_flags;
#endif
    struct hlist_bl_head s_roots; /* alternate root dentries for NFS */
    struct mount         *s_mounts; /* list of mounts; _not_ for fs use */
    struct block_device *s_bdev; /* can go away once we use an accessor

```

```

for @s_bdev_file */
    struct file          *s_bdev_file;
    struct backing_dev_info *s_bdi;
    struct mtd_info       *s_mtd;
    struct hlist_node      s_instances;
    unsigned int          s_quota_types; /* Bitmask of supported quota types */
    struct quota_info      s_dquot; /* Diskquota specific options */

    struct sb_writers      s_writers;

    /*
     * Keep s_fs_info, s_time_gran, s_fsnotify_mask, and
     * s_fsnotify_info together for cache efficiency. They are frequently
     * accessed and rarely modified.
     */
    void                  *s_fs_info; /* Filesystem private info */

    /* Granularity of c/m/atime in ns (cannot be worse than a second) */
    u32                   s_time_gran;
    /* Time limits for c/m/atime in seconds */
    time64_t              s_time_min;
    time64_t              s_time_max;
#ifdef CONFIG_FSNOTIFY
    u32                   s_fsnotify_mask;
    struct fsnotify_sb_info *s_fsnotify_info;
#endif

    char                  s_id[32]; /* Informational name */
    uuid_t                s_uuid; /* UUID */
    u8                    s_uuid_len; /* Default 16, possibly smaller for weird
filesystems */

    /* if set, fs shows up under sysfs at /sys/fs/$FSTYP/s_sysfs_name */
    char                  s_sysfs_name[UUID_STRING_LEN + 1];

    unsigned int          s_max_links;
    unsigned int          s_d_flags; /* default d_flags for dentries */

    /*
     * The next field is for VFS *only*. No filesystems have any business
     * even looking at it. You had been warned.
     */
    struct mutex          s_vfs_rename_mutex; /* Kludge */

    /*
     * Filesystem subtype. If non-empty the filesystem type field
     * in /proc/mounts will be "type.subtype"
     */
    const char            *s_subtype;

    const struct dentry_operations *__s_d_op; /* default d_op for
dentries */

    struct shrinker        *s_shrink; /* per-sb shrinker handle */

```

```

/* Number of inodes with nlink == 0 but still referenced */
atomic_long_t      s_remove_count;

/* Read-only state of the superblock is being changed */
int                s_readonly_remount;

/* per-sb errseq_t for reporting writeback errors via syncfs */
errseq_t s_wb_err;

/* AIO completions deferred from interrupt context */
struct workqueue_struct *s_dio_done_wq;
struct hlist_head      s_pins;

/*
 * Owning user namespace and default context in which to
 * interpret filesystem uids, gids, quotas, device nodes,
 * xattrs and security labels.
 */
struct user_namespace  *s_user_ns;

/*
 * The list_lru structure is essentially just a pointer to a table
 * of per-node lru lists, each of which has its own spinlock.
 * There is no need to put them into separate cachelines.
 */
struct list_lru         s_dentry_lru;
struct list_lru         s_inode_lru;
struct rcu_head          rcu;
struct work_struct       destroy_work;

struct mutex            s_sync_lock; /* sync serialisation lock */

/*
 * Indicates how deep in a filesystem stack this SB is
 */
int s_stack_depth;

/* s_inode_list_lock protects s_inodes */
spinlock_t             s_inode_list_lock ____cacheline_aligned_in_smp;
struct list_head        s_inodes; /* all inodes */

spinlock_t             s_inode_wblist_lock;
struct list_head        s_inodes_wb; /* writeback inodes */
long                   s_min_writeback_pages;

/* number of fserrors that are being sent to fsnotify/filesystems */
refcount_t             s_pending_errors;
}

```

Листинг 17 — struct super_block

struct inode

```

struct inode {
    umode_t      i_mode;
    unsigned short i_opflags;

```

```

    unsigned int    i_flags;
#ifdef CONFIG_FS_POSIX_ACL
    struct posix_acl *i_acl;
    struct posix_acl *i_default_acl;
#endif
    kuid_t          i_uid;
    kgid_t          i_gid;

    const struct inode_operations *i_op;
    struct super_block *i_sb;
    struct address_space *i_mapping;

#ifdef CONFIG_SECURITY
    void            *i_security;
#endif

    /* Stat data, not accessed from path walking */
    unsigned long    i_ino;
    /*
     * Filesystems may only read i_nlink directly. They shall use the
     * following functions for modification:
     *
     * (set|clear|inc|drop)_nlink
     * inode_(inc|dec)_link_count
     */
    union {
        const unsigned int i_nlink;
        unsigned int __i_nlink;
    };
    dev_t            i_rdev;
    loff_t           i_size;
    time64_t         i_atime_sec;
    time64_t         i_mtime_sec;
    time64_t         i_ctime_sec;
    u32              i_atime_nsec;
    u32              i_mtime_nsec;
    u32              i_ctime_nsec;
    u32              i_generation;
    spinlock_t       i_lock; /* i_blocks, i_bytes, maybe i_size */
    unsigned short    i_bytes;
    u8               i_blkbits;
    enum rw_hint      i_write_hint;
    blkcnt_t         i_blocks;

#ifdef __NEED_I_SIZE_ORDERED
    seqcount_t        i_size_seqcount;
#endif

    /* Misc */
    struct inode_state_flags i_state;
    /* 32-bit hole */
    struct rw_semaphore i_rwsem;

    unsigned long     dirtied_when; /* jiffies of first dirtying */
    unsigned long     dirtied_time_when;

```



```

struct hlist_node i_hash;
struct list_head i_io_list; /* backing dev IO list */
#ifdef CONFIG_CGROUP_WRITEBACK
struct bdi_writeback *i_wb; /* the associated cgroup wb */

/* foreign inode detection, see wbc_detach_inode() */
int i_wb_frn_winner;
u16 i_wb_frn_avg_time;
u16 i_wb_frn_history;
#endif
struct list_head i_lru; /* inode LRU list */
struct list_head i_sb_list;
struct list_head i_wb_list; /* backing dev writeback list */
union {
    struct hlist_head i_dentry;
    struct rcu_head i_rcu;
};
atomic64_t i_version;
atomic64_t i_sequence; /* see futex */
atomic_t i_count;
atomic_t i_dio_count;
atomic_t i_writecount;
#ifdef CONFIG_IMA || defined(CONFIG_FILE_LOCKING)
atomic_t i_readcount; /* struct files open RO */
#endif
union {
    const struct file_operations *i_fop; /* former ->i_op-
>default_file_ops */
    void (*free_inode)(struct inode *);
};
struct file_lock_context *i_flctx;
struct address_space i_data;
union {
    struct list_head i_devices;
    int i_linklen;
};
union {
    struct pipe_inode_info *i_pipe;
    struct cdev *i_cdev;
    char *i_link;
    unsigned i_dir_seq;
};

void *i_private; /* fs or device private pointer */
}

```

Листинг 18 — struct inode

struct inode_operations

```

struct inode_operations {
    struct dentry * (*lookup) (struct inode *, struct dentry *, unsigned
int);
    const char * (*get_link) (struct dentry *, struct inode *, struct
delayed_call *);
    int (*permission) (struct mnt_idmap *, struct inode *, int);
}

```

```

struct posix_acl * (*get_inode_acl)(struct inode *, int, bool);

int (*readlink) (struct dentry *, char __user *,int);

int (*create) (struct mnt_idmap *, struct inode *,struct dentry *,
               umode_t, bool);
int (*link) (struct dentry *,struct inode *,struct dentry *);
int (*unlink) (struct inode *,struct dentry *);
int (*symlink) (struct mnt_idmap *, struct inode *,struct dentry *,
               const char *);
struct dentry *(*mkdir) (struct mnt_idmap *, struct inode *,
                        struct dentry *, umode_t);
int (*rmdir) (struct inode *,struct dentry *);
int (*mknod) (struct mnt_idmap *, struct inode *,struct dentry *,
             umode_t,dev_t);
int (*rename) (struct mnt_idmap *, struct inode *, struct dentry *,
              struct inode *, struct dentry *, unsigned int);
int (*setattr) (struct mnt_idmap *, struct dentry *, struct iattr *);
int (*getattr) (struct mnt_idmap *, const struct path *,
               struct kstat *, u32, unsigned int);
ssize_t (*listxattr) (struct dentry *, char *, size_t);
int (*fiemap)(struct inode *, struct fiemap_extent_info *, u64 start,
             u64 len);
int (*update_time)(struct inode *inode, enum fs_update_time type,
                  unsigned int flags);
void (*sync_lazytime)(struct inode *inode);
int (*atomic_open)(struct inode *, struct dentry *,
                  struct file *, unsigned open_flag,
                  umode_t create_mode);
int (*tmpfile) (struct mnt_idmap *, struct inode *,
               struct file *, umode_t);
struct posix_acl *(*get_acl)(struct mnt_idmap *, struct dentry *,
                             int);
int (*set_acl)(struct mnt_idmap *, struct dentry *,
               struct posix_acl *, int);
int (*fileattr_set)(struct mnt_idmap *idmap,
                   struct dentry *dentry, struct file_kattr *fa);
int (*fileattr_get)(struct dentry *dentry, struct file_kattr *fa);
struct offset_ctx *(*get_offset_ctx)(struct inode *inode);
}

```

Листинг 19 — struct inode_operations

struct dentry

```

struct dentry {
    /* RCU lookup touched fields */
    unsigned int d_flags; /* protected by d_lock */
    seqcount_spinlock_t d_seq; /* per dentry seqlock */
    struct hlist_bl_node d_hash; /* lookup hash list */
    struct dentry *d_parent; /* parent directory */
    union {
        struct qstr __d_name; /* for use ONLY in fs/dcache.c */
        const struct qstr d_name;
    };
    struct inode *d_inode; /* Where the name belongs to - NULL is

```

```

        * negative */
union shortname_store d_shortname;
/* --- cacheline 1 boundary (64 bytes) was 32 bytes ago --- */

/* Ref lookup also touches following */
const struct dentry_operations *d_op;
struct super_block *d_sb; /* The root of the dentry tree */
unsigned long d_time; /* used by d_revalidate */
void *d_fsdata; /* fs-specific data */
/* --- cacheline 2 boundary (128 bytes) --- */
struct lockref d_lockref; /* per-dentry lock and refcount
    * keep separate from RCU lookup area if
    * possible!
    */

union {
    struct list_head d_lru; /* LRU list */
    wait_queue_head_t *d_wait; /* in-lookup ones only */
};
struct hlist_node d_sib; /* child of parent list */
struct hlist_head d_children; /* our children */
/*
    * d_alias and d_rcu can share memory
    */
union {
    struct hlist_node d_alias; /* inode alias list */
    struct hlist_bl_node d_in_lookup_hash; /* only for in-lookup ones */
    struct rcu_head d_rcu;
} d_u;
};

```

Листинг 20 — struct dentry

struct super_operations

```

struct super_operations {
    struct inode *(*alloc_inode)(struct super_block *sb);
    void (*destroy_inode)(struct inode *inode);
    void (*free_inode)(struct inode *inode);
    void (*dirty_inode)(struct inode *inode, int flags);
    int (*write_inode)(struct inode *inode, struct writeback_control *wbc);
    int (*drop_inode)(struct inode *inode);
    void (*evict_inode)(struct inode *inode);
    void (*put_super)(struct super_block *sb);
    int (*sync_fs)(struct super_block *sb, int wait);
    int (*freeze_super)(struct super_block *sb, enum freeze_holder who,
        const void *owner);
    int (*freeze_fs)(struct super_block *sb);
    int (*thaw_super)(struct super_block *sb, enum freeze_holder who,
        const void *owner);
    int (*unfreeze_fs)(struct super_block *sb);
    int (*statfs)(struct dentry *dentry, struct kstatfs *kstatfs);
    void (*umount_begin)(struct super_block *sb);

    int (*show_options)(struct seq_file *seq, struct dentry *dentry);
    int (*show_devname)(struct seq_file *seq, struct dentry *dentry);
};

```

```

int (*show_path)(struct seq_file *seq, struct dentry *dentry);
int (*show_stats)(struct seq_file *seq, struct dentry *dentry);
#ifdef CONFIG_QUOTA
ssize_t (*quota_read)(struct super_block *sb, int type, char *data,
                      size_t len, loff_t off);
ssize_t (*quota_write)(struct super_block *sb, int type,
                      const char *data, size_t len, loff_t off);
struct dquot __rcu **(*get_dquots)(struct inode *inode);
#endif
long (*nr_cached_objects)(struct super_block *sb,
                          struct shrink_control *sc);
long (*free_cached_objects)(struct super_block *sb,
                           struct shrink_control *sc);

void (*report_error)(const struct fserror_event *event);
};

```

Листинг 21 — struct super_operations

struct FILE

```
typedef struct _IO_FILE FILE;
```

Листинг 22 — struct FILE

struct IO_FILE

```

struct _IO_FILE {
    int _flags; /* High-order word is _IO_MAGIC; rest is flags. */
#define _IO_file_flags _flags

    /* The following pointers correspond to the C++ streambuf protocol. */
    /* Note: Tk uses the _IO_read_ptr and _IO_read_end fields directly. */
    char* _IO_read_ptr; /* Current read pointer */
    char* _IO_read_end; /* End of get area. */
    char* _IO_read_base; /* Start of putback+get area. */
    char* _IO_write_base; /* Start of put area. */
    char* _IO_write_ptr; /* Current put pointer. */
    char* _IO_write_end; /* End of put area. */
    char* _IO_buf_base; /* Start of reserve area. */
    char* _IO_buf_end; /* End of reserve area. */
    /* The following fields are used to support backing up and undo. */
    char *_IO_save_base; /* Pointer to start of non-current get area. */
    char *_IO_backup_base; /* Pointer to first valid character of backup
area */
    char *_IO_save_end; /* Pointer to end of non-current get area. */

    struct _IO_marker *_markers;

    struct _IO_FILE *_chain;

    int _fileno;
#ifdef 0
    int _blksize;
#else
    int _flags2;

```

```

#endif
_IO_off_t _old_offset; /* This used to be _offset but it's too small.
*/

#define __HAVE_COLUMN /* temporary */
/* 1+column number of pbase(); 0 is unknown. */
unsigned short _cur_column;
signed char _vtable_offset;
char _shortbuf[1];

/* char* _save_gptr; char* _save_egptr; */

_IO_lock_t *_lock;
#ifdef _IO_USE_OLD_IO_FILE
};

```

Листинг 23 — struct IO_FILE

struct stat

```

struct stat {
    dev_t      st_dev;          /* [XSI] ID of device containing file */
    ino_t      st_ino;          /* [XSI] File serial number */
    mode_t     st_mode;         /* [XSI] Mode of file (see below) */
    nlink_t    st_nlink;        /* [XSI] Number of hard links */
    uid_t      st_uid;          /* [XSI] User ID of the file */
    gid_t      st_gid;          /* [XSI] Group ID of the file */
    dev_t      st_rdev;         /* [XSI] Device ID */
#ifdef !defined(_POSIX_C_SOURCE) || defined(_DARWIN_C_SOURCE)
    struct timespec st_atimespec; /* time of last access */
    struct timespec st_mtimespec; /* time of last data modification */
    struct timespec st_ctimespec; /* time of last status change */
#else
    time_t      st_atime;        /* [XSI] Time of last access */
    long        st_atimensec;    /* nsec of last access */
    time_t      st_mtime;        /* [XSI] Last data modification time */
    long        st_mtimensec;    /* last data modification nsec */
    time_t      st_ctime;        /* [XSI] Time of last status change */
    long        st_ctimensec;    /* nsec of last status change */
#endif
    off_t      st_size;         /* [XSI] file size, in bytes */
    blkcnt_t   st_blocks;       /* [XSI] blocks allocated for file */
    blksize_t   st_blksize;     /* [XSI] optimal blocksize for I/O */
    __uint32_t  st_flags;        /* user defined flags for file */
    __uint32_t  st_gen;          /* file generation number */
    __int32_t   st_lspare;       /* RESERVED: DO NOT USE! */
    __int64_t   st_qspare[2];    /* RESERVED: DO NOT USE! */
};

```

Листинг 24 — struct stat

struct fs_struct

```

struct fs_struct {
    int users;
    seqlock_t seq;

```

```

int umask;
int in_exec;
struct path root, pwd;
}

```

Листинг 25 — struct fs_struct

struct file

```

/**
 * struct file - Represents a file
 * @f_lock: Protects f_ep, f_flags. Must not be taken from IRQ context.
 * @f_mode: FMODE_* flags often used in hotpaths
 * @f_op: file operations
 * @f_mapping: Contents of a cacheable, mappable object.
 * @private_data: filesystem or driver specific data
 * @f_inode: cached inode
 * @f_flags: file flags
 * @f_iocb_flags: iocb flags
 * @f_cred: stashed credentials of creator/opener
 * @f_owner: file owner
 * @f_path: path of the file
 * @__f_path: writable alias for @f_path; *ONLY* for core VFS and only
before
 *   the file gets open
 * @f_pos_lock: lock protecting file position
 * @f_pipe: specific to pipes
 * @f_pos: file position
 * @f_security: LSM security context of this file
 * @f_wb_err: writeback error
 * @f_sb_err: per sb writeback errors
 * @f_ep: link of all epoll hooks for this file
 * @f_task_work: task work entry point
 * @f_llist: work queue entriypoint
 * @f_ra: file's readahead state
 * @f_freeptr: Pointer used by SLAB_TYPESAFE_BY_RCU file cache (don't
touch.)
 * @f_ref: reference count
 */
struct file {
    spinlock_t      f_lock;
    fmode_t         f_mode;
    const struct file_operations *f_op;
    struct address_space *f_mapping;
    void            *private_data;
    struct inode     *f_inode;
    unsigned int     f_flags;
    unsigned int     f_iocb_flags;
    const struct cred *f_cred;
    struct fown_struct *f_owner;
    /* --- cacheline 1 boundary (64 bytes) --- */
    union {
        const struct path f_path;
        struct path      __f_path;
    };
    union {

```

```

    /* regular files (with FMODE_ATOMIC_POS) and directories */
    struct mutex    f_pos_lock;
    /* pipes */
    u64            f_pipe;
};
loff_t            f_pos;
#ifdef CONFIG_SECURITY
    void            *f_security;
#endif
/* --- cacheline 2 boundary (128 bytes) --- */
errseq_t          f_wb_err;
errseq_t          f_sb_err;
#ifdef CONFIG_EPOLL
    struct hlist_head *f_ep;
#endif
union {
    struct callback_head f_task_work;
    struct llist_node f_llist;
    struct file_ra_state f_ra;
    freeptr_t f_freeptr;
};
file_ref_t        f_ref;
/* --- cacheline 3 boundary (192 bytes) --- */
}

```

Листинг 26 — struct file

struct vfsmount

```

struct vfsmount {
    struct dentry *mnt_root; /* root of the mounted tree */
    struct super_block *mnt_sb; /* pointer to superblock */
    int mnt_flags;
    struct mnt_idmap *mnt_idmap;
}

```

Листинг 27 — struct vfsmount

struct file_operations

```

struct file_operations {
    struct module *owner;
    fop_flags_t fop_flags;
    loff_t (*llseek) (struct file *, loff_t, int);
    ssize_t (*read) (struct file *, char __user *, size_t, loff_t *);
    ssize_t (*write) (struct file *, const char __user *, size_t, loff_t *);
    ssize_t (*read_iter) (struct kiocb *, struct iov_iter *);
    ssize_t (*write_iter) (struct kiocb *, struct iov_iter *);
    int (*iopoll)(struct kiocb *kiocb, struct io_comp_batch *,
        unsigned int flags);
    int (*iterate_shared) (struct file *, struct dir_context *);
    __poll_t (*poll) (struct file *, struct poll_table_struct *);
    long (*unlocked_ioctl) (struct file *, unsigned int, unsigned long);
    long (*compat_ioctl) (struct file *, unsigned int, unsigned long);
    int (*mmap) (struct file *, struct vm_area_struct *);
    int (*open) (struct inode *, struct file *);
}

```



```

int (*flush) (struct file *, fl_owner_t id);
int (*release) (struct inode *, struct file *);
int (*fsync) (struct file *, loff_t, loff_t, int datasync);
int (*fasync) (int, struct file *, int);
int (*lock) (struct file *, int, struct file_lock *);
unsigned long (*get_unmapped_area)(struct file *, unsigned long,
unsigned long, unsigned long, unsigned long);
int (*check_flags)(int);
int (*flock) (struct file *, int, struct file_lock *);
ssize_t (*splice_write)(struct pipe_inode_info *, struct file *, loff_t
*, size_t, unsigned int);
ssize_t (*splice_read)(struct file *, loff_t *, struct pipe_inode_info
*, size_t, unsigned int);
void (*splice_eof)(struct file *file);
int (*setlease)(struct file *, int, struct file_lease **, void **);
long (*fallocate)(struct file *file, int mode, loff_t offset,
loff_t len);
void (*show_fdinfo)(struct seq_file *m, struct file *f);
#ifdef CONFIG_MMU
unsigned (*mmap_capabilities)(struct file *);
#endif
ssize_t (*copy_file_range)(struct file *, loff_t, struct file *,
loff_t, size_t, unsigned int);
loff_t (*remap_file_range)(struct file *file_in, loff_t pos_in,
struct file *file_out, loff_t pos_out,
loff_t len, unsigned int remap_flags);
int (*fadvise)(struct file *, loff_t, loff_t, int);
int (*uring_cmd)(struct io_uring_cmd *ioucmd, unsigned int issue_flags);
int (*uring_cmd_iopoll)(struct io_uring_cmd *, struct io_comp_batch *,
unsigned int poll_flags);
int (*mmap_prepare)(struct vm_area_desc *);
}

```

Листинг 28 — struct file_operations

Загрузка модуля

```

#include <linux/module.h>
#include <linux/init.h>

static int __init md_init(void)
{
    printk("Module loaded!\n");
    return 0;
}

static int __init md_init(void)
{
    struct task_struct *task = &init_task;
    do {
        printk(KERN_INFO "Task: pid: %d, comm: %s, ppid: %d, state: %ld,
flags: %x, prio: %d, policy: %d, root: %s, pwd: %s\n",
            task->pid, task->comm, task->parent->pid,
            task->state, task->flags, task->prio, task->policy,
            task->fs->root.dentry->d_iname, task->fs->pwd.dentry->d_iname);
    } while (0);
}

```



```

>d_name.name);
    } while ((task = next_task(task)) != &init_task);
    printk(KERN_INFO "Current: pid: %d, comm: %s, ppid: %d, state: %ld,
flags: %x, prio: %d, policy: %d, root: %s, pwd: %s\n",
        current->pid, current->comm, current->parent->pid,
        current->state, current->flags, current->prio, current->policy,
        current->fs->root.dentry->d_iname, current->fs->pwd.dentry-
>d_name.name);
    return 0;
}

static void __exit md_exit(void)
{
    printk("Module unloaded\n");
}

module_init(md_init);
module_exit(md_exit);
}

```

Листинг 29 — Загрузка модуля

printk flags

```

#define KERN_EMERG   KERN_SOH "0" /* system is unusable */
#define KERN_ALERT   KERN_SOH "1" /* action must be taken immediately */
#define KERN_CRIT    KERN_SOH "2" /* critical conditions */
#define KERN_ERR      KERN_SOH "3" /* error conditions */
#define KERN_WARNING  KERN_SOH "4" /* warning conditions */
#define KERN_NOTICE   KERN_SOH "5" /* normal but significant condition */
#define KERN_INFO     KERN_SOH "6" /* informational */
#define KERN_DEBUG    KERN_SOH "7" /* debug-level messages */
}

```

Листинг 30 — printk flags

Функции для работы с кэшем SLAB

```

struct kmem_cache *kmem_cache_create(const char *name, size_t size,
size_t offset,
                                unsigned long flags, void
(*constructor)(void *));

void kmem_cache_destroy(struct kmem_cache *cache);
void *kmem_cache_alloc(struct kmem_cache *cache, gfp_t flags);
void kmem_cache_free(struct kmem_cache *cache, void *obj);
}

```

Листинг 31 — Функции для работы с кэшем SLAB

simple_fill_super

```

int simple_fill_super(struct super_block *s, unsigned long magic,
const struct tree_descr *files)
{
    struct inode *inode;

```

```

struct dentry *dentry;
int i;

s->s_blocksize = PAGE_SIZE;
s->s_blocksize_bits = PAGE_SHIFT;
s->s_magic = magic;
s->s_op = &simple_super_operations;
s->s_time_gran = 1;

inode = new_inode(s);
if (!inode)
    return -ENOMEM;
/*
 * because the root inode is 1, the files array must not contain an
 * entry at index 1
 */
inode->i_ino = 1;
inode->i_mode = S_IFDIR | 0755;
simple_inode_init_ts(inode);
inode->i_op = &simple_dir_inode_operations;
inode->i_fop = &simple_dir_operations;
set_nlink(inode, 2);
s->s_root = d_make_root(inode);
if (!s->s_root)
    return -ENOMEM;
for (i = 0; !files->name || files->name[0]; i++, files++) {
    if (!files->name)
        continue;

    /* warn if it tries to conflict with the root inode */
    if (unlikely(i == 1))
        printk(KERN_WARNING "%s: %s passed in a files array"
            "with an index of 1!\n", __func__,
            s->s_type->name);

    dentry = d_alloc_name(s->s_root, files->name);
    if (!dentry)
        return -ENOMEM;
    inode = new_inode(s);
    if (!inode) {
        dput(dentry);
        return -ENOMEM;
    }
    inode->i_mode = S_IFREG | files->mode;
    simple_inode_init_ts(inode);
    inode->i_fop = files->ops;
    inode->i_ino = i;
    d_make_persistent(dentry, inode);
    dput(dentry);
}
return 0;
}

```